
MEZZINE_AI Research Assistant

Technical Architecture and Implementation Report

Project type: AI-powered research-paper assistant with RAG, multimodal ingestion, and
local/cloud AI modes

Author: Project Owner / Maintainer
Prepared by: MEZZINE MOHAMED
Version: 1.0
Date: June 24, 2026
Repository: `ai-research-assistant`

Contents

1	Executive Summary	1
2	Introduction	2
2.1	Context	2
2.2	Motivation	2
2.3	Objectives	2
2.4	Scope	3
3	Technical Stack	4
3.1	Technology Inventory	4
3.2	Dependency Gaps	6
4	System Architecture	7
4.1	Architectural Pattern	7
4.2	Component Diagram	7
4.3	Main Data Flow	7
4.3.1	PDF Upload and Ingestion	7
4.3.2	Question Answering	9
4.4	Module Dependencies	9
5	Project Structure	11
5.1	Annotated Directory Tree	11
5.2	Entry Points	12
6	Features and Functionalities	13
6.1	PDF Upload	13
6.2	Paper Knowledge Base Management	13

6.3	PDF Text and Image Extraction	13
6.4	Semantic Text Chunking	14
6.5	Embedding Generation	14
6.6	Vector Retrieval	14
6.7	Grounded Chat	15
6.8	Streaming Chat	15
6.9	Research Modes	15
6.10	Session Management	16
6.11	AI Mode Selection	16
6.12	Citations and PDF Viewer	17
7	Data Models and Database Design	18
7.1	Schema Overview	18
7.2	Relationships	18
7.3	Vector Storage	19
7.4	ER-Style Summary	19
8	API Reference	20
8.1	Endpoint Table	20
8.2	Main Request and Response Shapes	21
8.2.1	Chat Request	21
8.2.2	Paper Response	21
8.2.3	Streaming Response Events	21
9	Key Algorithms and Business Logic	22
9.1	Mode-Aware Provider Selection	22
9.2	Text Chunking	22
9.3	Multimodal Ingestion	23
9.4	Vector Retrieval	23
9.5	Session History Compression	24
10	Configuration and Environment	25
10.1	Configuration Source	25
10.2	Environment Variables	25
10.3	Configuration Notes	26

11 Security Considerations	27
11.1 Authentication and Authorization	27
11.2 CORS	27
11.3 Upload Risks	28
11.4 Frontend Rendering Risks	28
11.5 Debug Endpoint	28
11.6 Secrets	28
12 Testing Strategy	29
12.1 Current Test Assets	29
12.2 Recommended Test Strategy	29
12.3 How to Run Current Scripts	30
13 Deployment and Setup Guide	31
13.1 Local Development Setup	31
13.2 Private Mode Setup	32
13.3 Deployment Notes	32
14 Identified Issues and Recommendations	33
14.1 Highest Priority Issues	33
14.2 Code Quality Recommendations	34
14.3 Architecture Recommendations	35
15 Conclusion	36
References	37

List of Figures

4.1	High-level system architecture	8
4.2	Detailed ingestion pipeline	8
4.3	Streaming chat sequence	9
6.1	AI mode provider map	16
7.1	Database entity relationships	19

List of Tables

3.1	Primary technologies and their roles	4
7.1	Database entities	18
8.1	Public backend endpoints	20
10.1	Environment variables and defaults	25
14.1	Issues and recommendations	33

Chapter 1

Executive Summary

MEZZINE_AI Research Assistant is a full-stack AI research workspace that lets a user upload PDF papers, index their textual and visual content, and ask grounded questions over the uploaded knowledge base. The project implements a retrieval-augmented generation (RAG) pipeline using a FastAPI backend, PostgreSQL with pgvector, and a static browser frontend. It supports a cloud mode using Gemini, a hybrid mode using local embeddings with cloud chat generation, and a private mode designed around local embeddings and Ollama chat generation.

The system is best described as a modular monolith with a layered architecture. The frontend is a static single-page application (SPA), while the backend provides REST endpoints, streaming chat responses over newline-delimited JSON (NDJSON), database access, PDF ingestion, embedding generation, retrieval, prompt construction, and LLM provider selection.

At-a-glance Architecture Profile

Pattern: layered modular monolith. Primary workflow: upload PDF -> extract text/images -> chunk -> embed -> store vectors -> retrieve context -> generate grounded answer. Runtime modes: cloud, hybrid, and private. Main storage boundary: PostgreSQL with pgvector plus local paper/image files.

The current implementation is a capable MVP for personal or local research assistance. Its strongest elements are the separation between API routers, RAG modules, LLM providers, prompts, and CRUD utilities; its support for streaming responses; and its multimodal indexing path for extracted PDF images. The main production-readiness gaps are missing dependencies in `requirements.txt`, no authentication, permissive CORS, no upload size limit, a debug ingestion endpoint, global mutable AI mode state, and several frontend/backend contract mismatches that can break AI-mode loading and PDF citation viewing.

Chapter 2

Introduction

2.1 Context

Researchers often accumulate a collection of papers and need to query, compare, summarize, and synthesize them quickly. A RAG system is a practical architecture for this task: documents are chunked and embedded into a vector store; user questions are embedded; similar chunks are retrieved; and an LLM answers using the retrieved context.

This project applies that pattern to PDF research papers. It combines a web UI for upload and chat with backend services for ingestion, vector search, and LLM interaction. It also introduces privacy-oriented execution modes, allowing the user to choose how much document content is processed locally or sent to a cloud model.

2.2 Motivation

The project appears designed as a portfolio-quality AI engineering MVP. It demonstrates:

- RAG architecture with PostgreSQL and pgvector.
- FastAPI routing and request/response models.
- LLM provider abstraction across Gemini and Ollama.
- Local embedding support through SentenceTransformers.
- Streaming chat responses to a browser client.
- Session persistence and citation-based source traceability.
- A privacy-mode design that distinguishes cloud, hybrid, and local operation.

2.3 Objectives

The implemented system aims to:

- Accept PDF uploads from a browser UI.
- Extract text and images from each PDF page.
- Convert text and image descriptions into searchable vector chunks.
- Retrieve relevant chunks for user questions or research-analysis modes.
- Generate answers grounded only in retrieved sources.
- Persist chat sessions and message history.
- Display citations and allow source inspection in an embedded PDF viewer.

2.4 Scope

This report documents the source tree present in the analyzed workspace. It covers the backend, frontend, database schema, configuration, data flow, public API, feature set, known issues, and setup process. It does not claim that roadmap or proposal items are fully implemented unless corresponding source code exists.

Chapter 3

Technical Stack

3.1 Technology Inventory

Table 3.1: Primary technologies and their roles

Technology	Version Source	/ Role	Reason inferred from usage
Python	Not pinned	Backend language	FastAPI, PDF extraction, database access, and AI provider orchestration are implemented in Python.
FastAPI	0.110.0	Web API framework	Provides typed REST routes, request validation, file uploads, background tasks, and OpenAPI docs.
Uvicorn	0.27.1	ASGI server	Runs the FastAPI application during development and deployment.
Pydantic	2.6.3	Data validation	Defines request models such as chat requests and session title updates.
Pydantic Settings	2.2.1	Configuration	Loads environment variables and defaults through a single <code>Settings</code> object.
PostgreSQL	16 via Docker image	Relational database	Stores papers, chunks, sessions, and chat logs.
pgvector	pgvector/pgvector- v0.16.0	Vector search extension	Stores and compares embedding vectors using cosine distance.

Technology	Version Source	/ Role	Reason inferred from usage
psycopg	3.1.18	PostgreSQL client	Executes SQL queries and transactions from Python.
psycopg-pool	Missing from re- quirements	Connection pooling	Imported by <code>app/db.py</code> ; should be pinned for clean installs.
Google GenAI SDK	0.3.0	Gemini API client	Generates cloud embeddings, text answers, and cloud image descriptions.
Gemini	Configured model names	Cloud LLM and embeddings	Default cloud provider for chat and embeddings.
Ollama	Local HTTP API	Local chat provider	Used for private-mode text generation and Moon-dream image description.
SentenceTransformers	Optional, com- mented out	Local embeddings	Intended to generate local 768-dimensional embeddings. The import is currently eager.
PyTorch	Optional, com- mented out	Local model runtime	Used by SentenceTransformers and GPU/CPU device selection.
PyMuPDF	1.23.26	PDF parsing	Extracts page text and embedded images.
Pillow	10.2.0	Image loading	Sends extracted images to Gemini content generation.
HTTPX	0.27.0	HTTP client	Calls Ollama and test upload endpoints.
HTML/CSS/JavaScript	Browser native	Frontend SPA	Implements upload, chat, sessions, citations, and PDF viewer behavior without a build step.
marked	CDN latest URL	Markdown rendering	Renders LLM output in the chat interface.
MathJax	Version 3 CDN	Math rendering	Displays LaTeX-style math in generated answers.
PDF.js	3.11.174 CDN	Browser PDF viewer	Renders cited pages and highlights source snippets.
Font Awesome	6.4.0 CDN	Icons	Provides icons for UI controls and document markers.

Technology	Version Source	/ Role	Reason inferred from usage
Docker Compose	Compose file	Local infrastructure	Starts PostgreSQL with pgvector and schema initialization.

3.2 Dependency Gaps

The dependency file is close to the implemented stack, but a clean environment will likely fail without additional packages:

- `psycopg-pool` is required because `app/db.py` imports `ConnectionPool`.
- `python-multipart` is required by FastAPI file upload handling through `UploadFile` and `File`.
- `sentence-transformers` and `torch` are commented out as optional, but `app/llm/factory.py` imports `LocalEmbeddingProvider` eagerly, which imports both modules at import time.
- `pdfplumber` is imported by `backend/test_pdf.py`, but that file is an ad hoc script rather than application code.

Chapter 4

System Architecture

4.1 Architectural Pattern

The project follows a layered modular monolith architecture:

- **Presentation layer:** static frontend files in `frontend/`.
- **API layer:** FastAPI routers in `backend/app/api/`.
- **Application logic layer:** RAG ingestion, retrieval, prompts, provider selection, and session logic.
- **Provider layer:** Gemini, Ollama, and SentenceTransformers abstractions under `backend/app/llm/`.
- **Persistence layer:** PostgreSQL tables and pgvector columns accessed through `psycopg`.
- **Infrastructure layer:** Docker Compose starts the database and loads `backend/init.sql`.

It is not a microservice architecture. The backend is one FastAPI process with internally separated modules. The database is the only separate runtime service defined by the project.

4.2 Component Diagram

4.3 Main Data Flow

4.3.1 PDF Upload and Ingestion

1. The user uploads a PDF through the frontend drop zone or file picker.
2. The frontend sends `POST /papers/upload` with multipart form data.
3. The backend checks the file extension, saves the PDF into `data/papers/`, and creates or reuses a paper record.

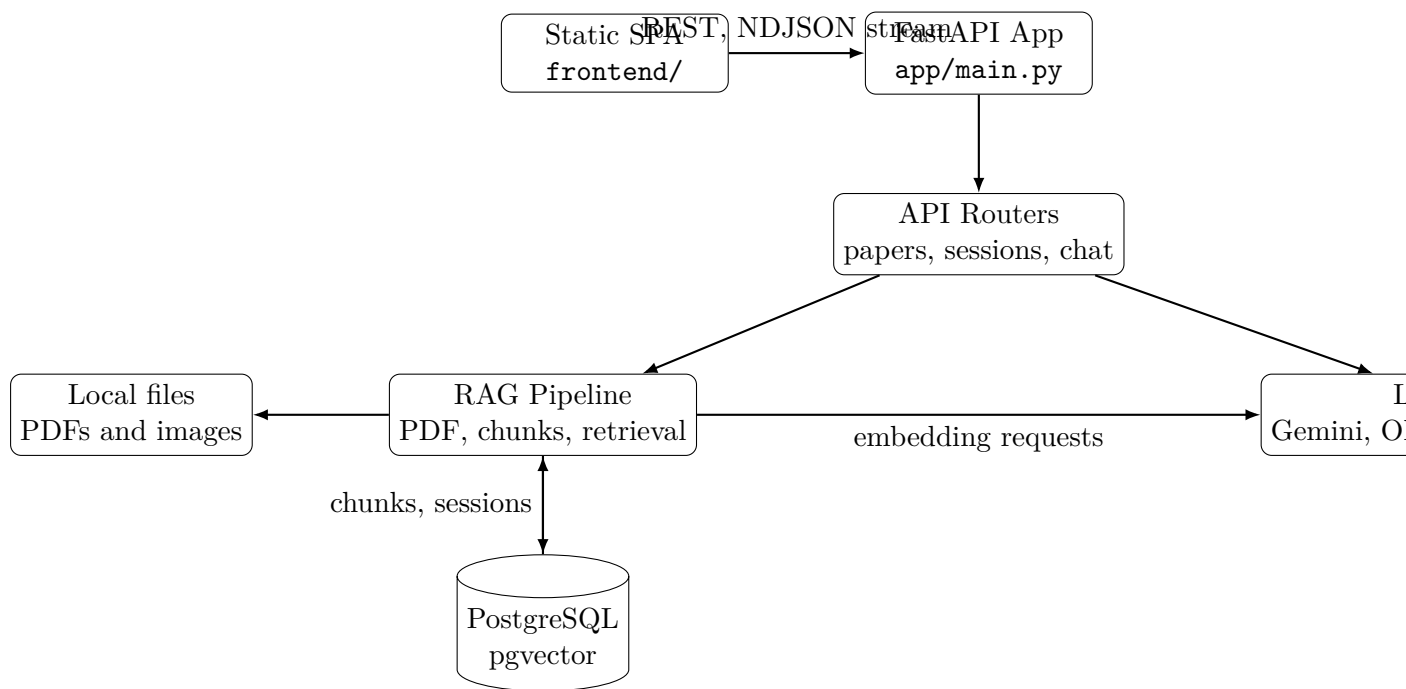


Figure 4.1: High-level system architecture

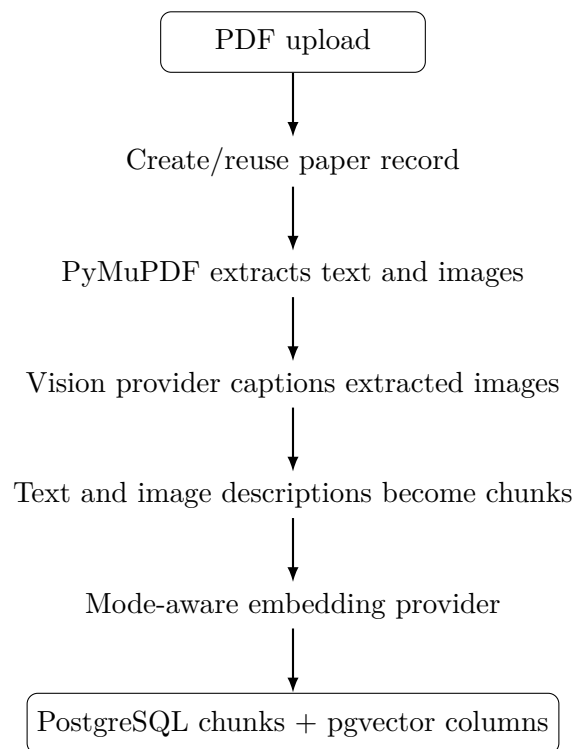


Figure 4.2: Detailed ingestion pipeline

4. A FastAPI background task runs `ingest_pdf()`.
5. `extract_pdf_pages()` uses PyMuPDF to extract page text and embedded images.
6. Text is chunked by `chunk_text()` using paragraph-first, sentence-fallback logic with overlap.
7. Extracted images are described through `generate_image_description()` and indexed as image-description chunks.
8. The selected embedding provider embeds each chunk.
9. Chunks and vectors are inserted into PostgreSQL.

4.3.2 Question Answering

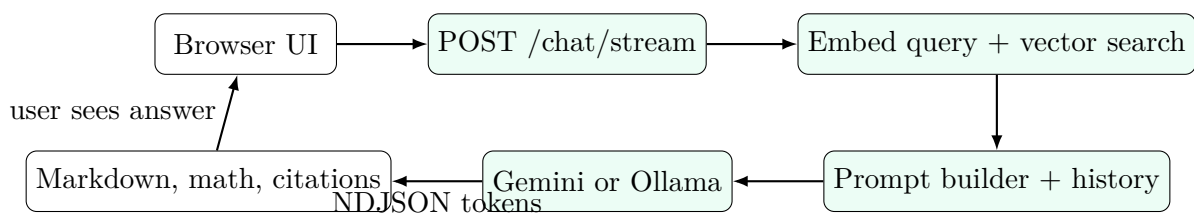


Figure 4.3: Streaming chat sequence

1. The user asks a question or triggers a research mode.
2. The frontend sends `POST /chat/stream` with session ID, question, selected paper IDs, and mode.
3. The backend embeds the question with the active embedding provider.
4. `retrieve_context()` searches the active vector column using pgvector cosine distance.
5. The prompt builder formats the question, retrieved chunks, mode instructions, and recent chat history.
6. The active chat provider generates an answer through Gemini or Ollama.
7. The streaming endpoint emits a source event followed by token events as NDJSON.
8. The frontend renders Markdown incrementally, displays citation chips, and saves the exchange through the backend.

4.4 Module Dependencies

- `app/main.py` configures the FastAPI app, CORS, migrations, routers, static mounts, mode endpoints, and frontend serving.
- `app/api/papers.py` depends on `app.rag.ingest`, `app.crud_papers`, `app.db`, and configuration.

- `app/api/chat.py` depends on retrieval, session history, and chat provider selection.
- `app/rag/ingest.py` depends on PDF loading, chunking, embeddings, image description, and database access.
- `app/rag/retrieval.py` depends on the embedding provider, active settings, and pgvector SQL.
- `app/llm/factory.py` maps runtime mode settings to concrete provider classes.
- `app/crud_sessions.py` and `app/crud_papers.py` isolate most direct persistence operations.

Chapter 5

Project Structure

5.1 Annotated Directory Tree

Listing 5.1: Project directory tree

ai-research-assistant/	
.env.example	Environment template with GEMINI_API_KEY placeholder
.gitignore	Ignores secrets, virtualenvs, caches, IDE files, and
data	
docker-compose.yml	PostgreSQL + pgvector service
README.md	Setup notes, partially stale OpenAI references
privacy_mode_local_llm_feature.md	
	Local/private AI mode design specification
PROJECT_CRITIC_REVIEW.md	Local review document with known issues
docs/	
architecture.md	Short architecture sketch
project_report.tex	This generated technical report
backend/	
requirements.txt	Backend dependencies
init.sql	Initial pgvector schema
test_*.py	Ad hoc diagnostic scripts
app/	
main.py	FastAPI app, routers, static mounts, settings
endpoints	
db.py	Connection pool, transaction helper, startup
migrations	
crud_papers.py	Paper listing and deletion
crud_sessions.py	Session CRUD and chat log persistence
api/	
__init__.py	
papers.py	Paper upload/list/delete routes
sessions.py	Session CRUD and chat history routes
chat.py	Chat and streaming chat routes
core/	
config.py	Pydantic settings

llm/	
base.py	Provider interfaces
factory.py	Mode-aware provider factory
gemini_provider.py	Gemini embeddings and chat generation
ollama_provider.py	Ollama local chat generation
local_embeddings.py	SentenceTransformers embedding provider
prompts.py	Mode-specific prompt templates
vision.py	Image description through Gemini or Moondream
rag/	
pdf_loader.py	PyMuPDF text and image extraction
chunker.py	Paragraph/sentence-aware text chunking
ingest.py	PDF ingestion, image captioning, embedding, insert
retrieval.py	Vector search and context assembly
frontend/	
index.html	Static SPA layout
styles.css	Dark UI styling
app.js	Upload, sessions, chat, mode switching, citations
pdf_viewer.js	PDF.js rendering and snippet highlighting
README.md	Placeholder frontend note
test.txt	Residual test file
data/	
papers/	Uploaded PDFs, ignored by git
images/	Extracted PDF images, ignored by git

5.2 Entry Points

- **Backend application:** backend/app/main.py, run with `uvicorn app.main:app --reload` from the backend/ directory.
- **Frontend application:** frontend/index.html, served by FastAPI at GET / and by the static mount at the root path.
- **Database initialization:** backend/init.sql, mounted into the PostgreSQL container by Docker Compose.
- **Infrastructure:** docker-compose.yml, defining the pgvector/pgvector:pg16 database service on host port 5433.

Chapter 6

Features and Functionalities

6.1 PDF Upload

Files: `frontend/app.js`, `backend/app/api/papers.py`, `backend/app/rag/ingest.py`

The frontend supports drag-and-drop and file-picker upload. The backend endpoint `POST /papers/upload` accepts a multipart file and rejects filenames that do not end in `.pdf`. The file is saved to `settings.upload_dir`, a paper record is created or reused by title, and ingestion is scheduled with FastAPI `BackgroundTasks`. The endpoint returns immediately with a paper ID and title.

Internally, this keeps the request path responsive while the heavier PDF parsing and embedding work runs after the response. The implementation currently reads the whole upload into memory with `await file.read()`, which is simple but risky for large PDFs.

6.2 Paper Knowledge Base Management

Files: `frontend/app.js`, `backend/app/api/papers.py`, `backend/app/crud_papers.py`

The frontend lists uploaded papers with checkboxes and delete icons. The backend exposes:

- `GET /papers` to list papers for the active knowledge-base mode.
- `DELETE /papers/{paper_id}` to delete a paper and cascade associated chunks.

The list operation filters papers by `papers.ai_mode`. In the current upload implementation, however, new papers are inserted without setting `ai_mode`; the database default is `local`. This creates a mode-consistency bug for cloud uploads.

6.3 PDF Text and Image Extraction

Files: `backend/app/rag/pdf_loader.py`, `backend/app/llm/vision.py`

`extract_pdf_pages()` opens the PDF with PyMuPDF, extracts text from each page, and iterates over embedded images. Images smaller than 5000 bytes are ignored to skip tiny decorative artifacts. Larger images are stored under `data/images/` with a filename containing the paper stem, page number, image index, and a short UUID.

During ingestion, each stored image is sent to `generate_image_description()`. Private mode uses the Ollama `moondream` model with base64 image input; other modes use Gemini vision. The generated description becomes a textual chunk with an `[IMAGE]` prefix and retains the image path for citation rendering.

6.4 Semantic Text Chunking

Files: `backend/app/rag/chunker.py`

The chunker splits text first by paragraph boundaries, then by sentence boundaries for oversized paragraphs, and finally by fixed-size windows when a single sentence exceeds the maximum chunk size. The defaults are `max_chunk_size = 1500` characters and `overlap = 200` characters.

This is an MVP-level semantic chunker. It improves over naive fixed slicing by respecting paragraphs and sentences where possible, but it is still character-based rather than token-based and does not handle table structure, equations, or multi-column reading order.

6.5 Embedding Generation

Files: `backend/app/llm/base.py`, `backend/app/llm/factory.py`,
`backend/app/llm/gemini_provider.py`, `backend/app/llm/local_embeddings.py`

Embedding providers implement the `EmbeddingProvider` interface. The factory chooses providers based on `settings.ai_mode`:

- **Cloud mode:** Gemini embeddings.
- **Hybrid mode:** local SentenceTransformers embeddings.
- **Private mode:** local SentenceTransformers embeddings.

Ingestion stores cloud-mode embeddings in `chunks.embedding_cloud` and local/private/hybrid embeddings in `chunks.embedding_local`. The local column is fixed at 768 dimensions in the schema, matching the configured `sentence-transformers/all-mpnet-base-v2` model.

6.6 Vector Retrieval

Files: `backend/app/rag/retrieval.py`, `backend/init.sql`

`retrieve_context()` embeds the user query, chooses the active vector column, and runs a pgvector cosine-similarity query. It computes:

Listing 6.1: Similarity expression used for retrieval

```
1 - (chunks.embedding_local <=> %s::vector) AS similarity
```

The query filters out rows with similarity at or below 0.05, orders by similarity descending, and limits results by `top_k`. If paper IDs are provided, it restricts retrieval to those selected papers.

6.7 Grounded Chat

Files: `backend/app/api/chat.py`, `backend/app/llm/prompts.py`,
`backend/app/llm/gemini_provider.py`, `backend/app/llm/ollama_provider.py`

The standard chat endpoint takes a question, retrieves source chunks, builds an academic prompt, and asks the selected chat provider to answer only using retrieved sources. The prompt requires inline citation formatting such as (Source X, page Y).

The backend also adds recent session history for ordinary chat mode. It fetches the last five messages and truncates assistant messages longer than 500 characters, limiting context growth.

6.8 Streaming Chat

Files: `backend/app/api/chat.py`, `frontend/app.js`

POST `/chat/stream` returns a `StreamingResponse` with media type `application/x-ndjson`. The stream first yields a `Sources` object, then emits token objects:

Listing 6.2: Streaming response event shapes

```
{"type": "sources", "data": [...]}\n{"type": "token", "data": "partial answer text"}
```

The frontend reads the stream through `ReadableStream.getReader()`, parses newline-delimited JSON, renders partial Markdown using `marked`, and appends citation chips once sources arrive.

6.9 Research Modes

Files: `frontend/index.html`, `frontend/app.js`, `backend/app/llm/prompts.py`

The UI exposes seven research-mode buttons:

- Summarize
- Methodology
- Contributions
- Limitations

- Gap Finder
- Compare
- Literature Review

When a mode is selected without a question, the backend substitutes a broad retrieval query and increases `top_k` to 10. The prompt module then applies the corresponding task-specific instructions.

6.10 Session Management

Files: `frontend/app.js`, `backend/app/api/sessions.py`, `backend/app/crud_sessions.py`

The project supports persistent chat sessions:

- GET `/sessions` lists sessions.
- POST `/sessions` creates a new session titled **New Chat**.
- GET `/sessions/{session_id}/chat` returns ordered chat history.
- PUT `/sessions/{session_id}` updates the title.
- DELETE `/sessions/{session_id}` deletes the session and cascades chat logs.

After a user asks the first real question, the backend updates the title to the first 50 characters of the question.

6.11 AI Mode Selection

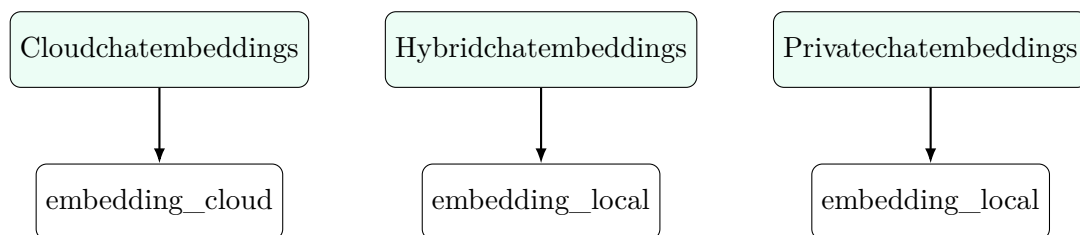


Figure 6.1: AI mode provider map

Files: `backend/app/main.py`, `frontend/app.js`, `backend/app/llm/factory.py`

The backend exposes runtime AI mode endpoints:

- GET `/settings/ai-mode`
- PUT `/settings/ai-mode`

Valid modes are `cloud`, `hybrid`, and `private`. The provider factory uses the active mode to decide which chat and embedding implementations to instantiate.

The current implementation mutates the process-global settings object. This is acceptable for a single-user local prototype but not safe for concurrent users or background ingestion jobs. There is also a frontend contract bug: the backend returns `\{"mode": "..."\}`, while the frontend reads `settings.ai_mode`.

6.12 Citations and PDF Viewer

Files: `frontend/app.js`, `frontend/pdf_viewer.js`, `backend/app/main.py`

Answers include citations derived from retrieved chunks. The frontend renders each source as a chip containing paper title, page number, similarity tooltip, and optionally an image thumbnail. Clicking a citation opens the PDF viewer pane implemented with `PDF.js`.

The viewer loads a PDF from `/files/papers/{filename}`, renders the cited page, extracts text positions, and highlights the closest text item matching the source snippet. The design is useful, but the current citation payload uses `paper_title` rather than the stored file path or filename. Since upload titles are title-cased stems without the `.pdf` suffix, many PDF viewer links will fail.

Chapter 7

Data Models and Database Design

7.1 Schema Overview

The database schema is defined in `backend/init.sql`. Startup migrations in `app/db.py` add session, multimodal, and mode-related columns if they are missing.

Table 7.1: Database entities

Table	Primary columns	Purpose
<code>papers</code>	<code>id</code> , <code>title</code> , <code>file_path</code> , <code>ai_mode</code> , <code>created_at</code>	Stores uploaded document metadata and the local filesystem path.
<code>chunks</code>	<code>id</code> , <code>paper_id</code> , <code>page_number</code> , <code>content</code> , <code>image_path</code> , <code>embedding_cloud</code> , <code>embedding_local</code> , <code>created_at</code>	Stores text chunks and image-description chunks with mode-specific embeddings.
<code>sessions</code>	<code>id</code> , <code>title</code> , <code>created_at</code> , <code>updated_at</code>	Stores chat conversation containers.
<code>chat_logs</code>	<code>id</code> , <code>session_id</code> , <code>role</code> , <code>content</code> , <code>sources</code> , <code>model_used</code> , <code>created_at</code>	Stores user and assistant messages plus assistant source metadata.

7.2 Relationships

- `papers.id` has a one-to-many relationship with `chunks.paper_id`.
- `chunks.paper_id` uses `ON DELETE CASCADE`, so deleting a paper deletes its chunks.

- `sessions.id` has a one-to-many relationship with `chat_logs.session_id`.
- `chat_logs.session_id` uses `ON DELETE CASCADE`, so deleting a session deletes its messages.

7.3 Vector Storage

The schema separates cloud and local embeddings:

- `embedding_cloud` vector: no fixed dimension, allowing flexibility for cloud embedding dimensions but preventing a normal fixed-dimension ivfflat index.
- `embedding_local` vector(768): fixed dimension for `all-mpnet-base-v2`, indexed with ivfflat cosine operations.

This design avoids mixing incompatible embedding dimensions in one column. The tradeoff is that cloud retrieval may rely on sequential vector search unless a dimension-specific cloud column or index is introduced.

7.4 ER-Style Summary

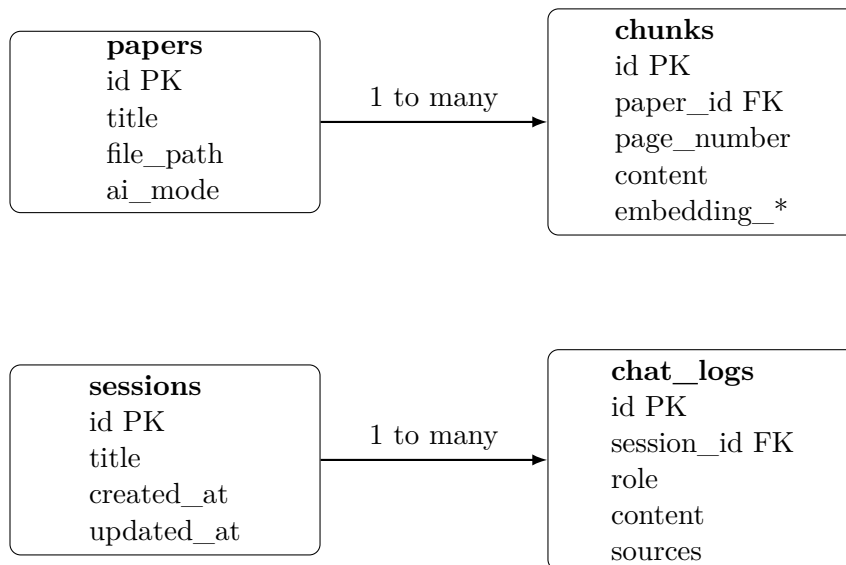


Figure 7.1: Database entity relationships

Chapter 8

API Reference

8.1 Endpoint Table

Table 8.1: Public backend endpoints

Method	Route	Purpose	Auth
GET	/	Serves <code>frontend/index.html</code> if available.	No
GET	/settings/ai-mode	Returns the current AI mode. Response key is currently <code>mode</code> .	No
PUT	/settings/ai-mode	Updates global AI mode to <code>cloud</code> , <code>hybrid</code> , or <code>private</code> .	No
GET	/papers	Lists papers for the active knowledge-base mode.	No
POST	/papers/upload	Uploads a PDF and starts background ingestion.	No
DELETE	/papers/{paper_id}	Deletes a paper and associated chunks.	No
GET	/sessions	Lists chat sessions.	No
POST	/sessions	Creates a new chat session.	No
GET	/sessions/{session_id}/chat	Returns chat history for a session.	No
PUT	/sessions/{session_id}	Updates a session title.	No
DELETE	/sessions/{session_id}	Deletes a session and associated chat logs.	No
POST	/chat	Performs non-streaming RAG answer generation.	No
POST	/chat/stream	Streams sources and answer tokens as NDJSON.	No
GET	/files/papers/{path}	Serves uploaded PDFs from the paper directory.	No

Method	Route	Purpose	Auth
GET	/images/{path}	Serves extracted images.	No
GET	/test-ingest	Debug endpoint that ingests the latest PDF and returns tracebacks on failure.	No

8.2 Main Request and Response Shapes

8.2.1 Chat Request

Listing 8.1: ChatRequest model

```
class ChatRequest(BaseModel):
    session_id: int
    question: Optional[str] = None
    top_k: int = 5
    paper_ids: Optional[List[int]] = None
    mode: str = "chat"
```

8.2.2 Paper Response

Listing 8.2: PaperResponse model

```
class PaperResponse(BaseModel):
    id: int
    title: str
    created_at: str
```

8.2.3 Streaming Response Events

Listing 8.3: NDJSON streaming payloads

```
{"type": "sources", "data": [
  {
    "paper_title": "Example Paper",
    "page_number": 3,
    "similarity": 0.8123,
    "content": "Retrieved source text",
    "image_path": null
  }
]}
{"type": "token", "data": "The answer begins..."}
```

Chapter 9

Key Algorithms and Business Logic

9.1 Mode-Aware Provider Selection

The factory module centralizes how runtime AI mode maps to provider implementations.

Listing 9.1: Provider selection logic

```
def get_chat_provider() -> ChatProvider:
    mode = settings.ai_mode.lower()

    if mode == "private":
        provider_name = settings.local_chat_provider
    elif mode == "hybrid":
        provider_name = settings.cloud_chat_provider
    else:
        provider_name = settings.cloud_chat_provider

    provider_class = _chat_providers.get(provider_name, GeminiChatProvider)
    return provider_class()
```

Business meaning:

- Cloud mode sends embeddings and final answer generation to Gemini.
- Hybrid mode indexes locally but still sends retrieved chunks to Gemini for answer generation.
- Private mode keeps both embeddings and chat generation local, assuming local dependencies and Ollama are installed.

9.2 Text Chunking

Listing 9.2: Paragraph-first chunking strategy

```
paragraphs = re.split(r'\n\s*\n', text)
```

```

for para in paragraphs:
    para = para.strip()
    if not para:
        continue

    if len(current_chunk) + len(para) + 2 <= max_chunk_size:
        current_chunk += para + "\n\n"
    else:
        if current_chunk:
            chunks.append(current_chunk.strip())
            overlap_text = current_chunk[-overlap:]
            current_chunk = overlap_text + "\n\n"

```

The chunker tries to preserve semantic boundaries by preferring paragraphs. It only falls back to sentence splitting or hard fixed-size windows when necessary. This is appropriate for an MVP, but production RAG would usually chunk by tokens and may use layout-aware PDF parsing.

9.3 Multimodal Ingestion

Listing 9.3: Image descriptions are converted into retrievable chunks

```

if page.get("images"):
    for img_path in page["images"]:
        description = generate_image_description(img_path, active_mode)
        if description:
            chunk_content = f"[IMAGE] Description: {description}"
            all_chunks_info.append({
                "page_number": page["page_number"],
                "content": chunk_content,
                "image_path": img_path,
            })

```

This converts chart, diagram, and figure content into text embeddings, allowing the retrieval system to surface image-derived evidence through the same vector search path as normal text.

9.4 Vector Retrieval

Listing 9.4: Active vector-column retrieval

```

active_mode = settings.ai_mode.lower()
target_column = "embedding_cloud" if active_mode == "cloud" else "embedding_local"

query = f"""
    SELECT chunk_id, paper_title, page_number, content, image_path, similarity
    FROM (
        SELECT

```

```
        chunks.id AS chunk_id,  
        papers.title AS paper_title,  
        chunks.page_number,  
        chunks.content,  
        chunks.image_path,  
        1 - (chunks.{target_column} <=> %s::vector) AS similarity  
FROM chunks  
JOIN papers ON papers.id = chunks.paper_id  
WHERE chunks.{target_column} IS NOT NULL  
) AS sim_query  
WHERE similarity > 0.05  
ORDER BY similarity DESC  
LIMIT %s  
"""
```

The selected column is not directly user-controlled; it is derived from validated application mode. The query uses parameter binding for the vector value and limit. The similarity threshold is very low and may admit weak matches.

9.5 Session History Compression

The session module includes recent conversation history in chat prompts but truncates long assistant answers:

Listing 9.5: Recent history truncation

```
recent = messages[-limit:]  
for msg in recent:  
    role = "User" if msg["role"] == "user" else "Assistant"  
    content = msg["content"]  
    if len(content) > 500:  
        content = content[:500] + "... [truncated]"  
    history_str += f"{role}: {content}\n\n"
```

This is a simple but practical guard against unbounded prompt growth.

Chapter 10

Configuration and Environment

10.1 Configuration Source

Configuration is centralized in `backend/app/core/config.py`. Pydantic Settings reads from environment variables and from `../.env` or `.env`, depending on the process working directory. The default run instructions start Uvicorn from `backend/`, so `../.env` points to the repository root.

10.2 Environment Variables

Table 10.1: Environment variables and defaults

Variable	Default / Required	Purpose
GEMINI_API_KEY	Required	API key for Gemini embeddings, chat, and cloud vision. Present as a placeholder in <code>.env.example</code> .
DATABASE_URL	postgresql://postgres:postgres@localhost:5433/research	PostgreSQL local host connection. Matches Docker Compose host port 5433.
UPLOAD_DIR	../data/papers	Directory where uploaded PDF files are stored.
AI_MODE	cloud	Active AI mode. Valid values in the API are <code>cloud</code> , <code>hybrid</code> , and <code>private</code> .
CLOUD_CHAT_PROVIDER	gemini	Provider key for cloud chat generation.
CLOUD_CHAT_MODEL	gemini-1.5-flash	Gemini model used for text answers.
CLOUD_EMBEDDING_PROVIDER	gemini	Provider key for cloud embeddings.
CLOUD_EMBEDDING_MODEL	text-embedding-004	Gemini embedding model.
LOCAL_CHAT_PROVIDER	ollama	Provider key for private-mode chat.

Variable	Default / Required	Purpose
LOCAL_CHAT_MODEL	llama3.1:8b	Ollama model used for private-mode chat generation.
OLLAMA_BASE_URL	http://localhost:11434	Base URL for the local Ollama HTTP API.
LOCAL_EMBEDDING_PROVIDER	sentence-transformers	Provider key for local embeddings.
LOCAL_EMBEDDING_MODEL	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	Selected OpenAI GPT-4o model used for 768-dimensional local embeddings.

10.3 Configuration Notes

- The README still references `OPENAI_API_KEY`, but the current code uses Gemini and does not import an OpenAI client.
- The README command `cp .env .env` appears to be a typo. It should copy from `.env.example` to `.env`.
- Because `gemini_api_key` has no default, importing settings fails unless `GEMINI_API_KEY` exists in the environment or an env file.
- Runtime AI mode is mutable through an endpoint, but it is not persisted to disk or per session.

Chapter 11

Security Considerations

Security Posture Summary

The current security posture is local-development only. The most important blockers for hosted use are missing authentication, permissive CORS, unrestricted file upload size, unsanitized frontend rendering, public static file access, and the exposed debug ingestion endpoint.

11.1 Authentication and Authorization

No authentication or authorization is implemented. All endpoints, including upload, delete, chat, static file serving, and the debug ingestion endpoint, are publicly callable by anyone who can reach the FastAPI process.

This is acceptable only for local development. A deployed version needs at least authentication, per-user resource scoping, and authorization checks for paper/session deletion.

11.2 CORS

`app/main.py` configures:

Listing 11.1: Current CORS configuration

```
allow_origins=["*"],
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
```

This is permissive. If the application is ever exposed beyond localhost, allowed origins should be restricted to the actual frontend origins, and credentials should be enabled only if needed.

11.3 Upload Risks

The upload endpoint currently validates only the filename extension. Key risks:

- No upload size limit; the whole file is read into memory.
- No MIME type or magic-byte validation.
- Original filenames are used for local paths, which can create collisions or path-safety concerns if not normalized.
- PDF parsing and image extraction are CPU and memory intensive.

11.4 Frontend Rendering Risks

The frontend uses `innerHTML` and renders Markdown from model output through `marked` without visible sanitization. It also inserts user messages and source metadata into HTML strings. This creates cross-site scripting risk if untrusted content reaches those paths. Generated answers, PDF text, filenames, and paper titles should be escaped or sanitized before insertion into the DOM.

11.5 Debug Endpoint

GET `/test-ingest` is exposed in `app/main.py`. It forces `settings.ai_mode = "hybrid"`, ingests the latest PDF, and returns tracebacks on error. This endpoint should be removed or protected behind an explicit debug setting before any non-local deployment.

11.6 Secrets

`.env.example` contains a placeholder rather than a real key, and `.gitignore` excludes `.env` and `backend/.env`. This is good. The Docker Compose database credentials are hard-coded development credentials and should not be reused in production.

Chapter 12

Testing Strategy

12.1 Current Test Assets

The repository contains multiple `backend/test_*.py` files. They are mostly manual diagnostic scripts rather than automated tests. Examples include:

- Database inspection scripts such as `test_db.py` through `test_db15.py`.
- Ingestion smoke script `test_ingest.py`.
- Upload smoke script `test_upload.py`.
- PDF extraction inspection scripts `test_pdf.py` and `test_extract.py`.

These scripts print database state or call local endpoints. They do not define repeatable assertions, fixtures, mocks, or CI-ready test commands.

12.2 Recommended Test Strategy

- Add a `backend/tests/` directory using `pytest`.
- Unit-test `chunk_text()` for paragraph, sentence, overlap, and long-sentence cases.
- Unit-test provider factory behavior for `cloud`, `hybrid`, and `private` modes.
- Use FastAPI `TestClient` for route tests with mocked providers and database access.
- Integration-test schema creation with a disposable PostgreSQL container.
- Add retrieval tests with deterministic fake embeddings.
- Add frontend smoke tests for upload, session creation, streaming response handling, and citation-click behavior.
- Add security tests for invalid upload types, oversized files, missing session IDs, and unauthorized access once auth is introduced.

12.3 How to Run Current Scripts

From `backend/`, after dependencies, database, and environment variables are configured:

Listing 12.1: Manual diagnostic script examples

```
python test_ingest.py
python test_upload.py
python test_db.py
```

These should be treated as local diagnostics, not as a formal test suite.

Chapter 13

Deployment and Setup Guide

13.1 Local Development Setup

1. Create the environment file at the repository root:

```
cp .env.example .env
```

2. Edit `.env` and set at least:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

3. Start PostgreSQL with pgvector:

```
docker compose up -d db
```

4. Create and activate a Python virtual environment:

```
cd backend
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS/Linux
source .venv/bin/activate
```

5. Install dependencies:

```
pip install -r requirements.txt
```

6. Install currently missing runtime dependencies until `requirements.txt` is fixed:

```
pip install psycopg-pool python-multipart
# Required for hybrid/private mode with the current eager imports:
pip install sentence-transformers torch
```

7. Run the backend:

```
uvicorn app.main:app --reload
```

8. Open the app:

```
http://127.0.0.1:8000/
```

13.2 Private Mode Setup

Private mode requires Ollama and local models:

1. Install and start Ollama.
2. Pull the configured chat model:

```
ollama pull llama3.1:8b
```

3. Pull the vision model used for private image descriptions:

```
ollama pull moondream
```

4. Ensure `OLLAMA_BASE_URL` points to the running Ollama server.
5. Switch mode through the UI or with PUT `/settings/ai-mode`.

13.3 Deployment Notes

For production or shared deployment, the project needs additional hardening before use:

- Add authentication and per-user authorization.
- Lock CORS to known frontend origins.
- Remove or guard `/test-ingest`.
- Add upload size limits and content validation.
- Use managed secrets and production database credentials.
- Move runtime mode from global mutable settings to per-request, per-session, or per-paper state.
- Serve frontend assets from a stable origin and pin CDN dependencies or self-host them.

Chapter 14

Identified Issues and Recommendations

14.1 Highest Priority Issues

Table 14.1: Issues and recommendations

Issue	Impact	Recommendation
Missing runtime dependencies	Clean installs can fail before the app starts.	Add <code>psycopg-pool</code> , <code>python-multipart</code> , and either include or lazily import local-mode dependencies.
Global mutable <code>settings.ai_mode</code>	Concurrent requests and background ingestion can use inconsistent modes.	Capture mode at upload/chat time and store mode on papers/sessions rather than mutating a singleton.
Paper <code>ai_mode</code> not set during upload	Cloud-mode paper listing and retrieval selection can break.	Insert <code>ai_mode</code> explicitly when creating a paper; consider storing embedding mode per chunk.
Frontend AI-mode response mismatch	Mode dropdown may initialize with <code>undefined</code> .	Change frontend to read <code>mode</code> or backend to return <code>ai_mode</code> ; use one contract consistently.
Citation PDF filename mismatch	Clicking citations may fail to load the PDF.	Return <code>file_path</code> or stored filename in source metadata instead of using title as filename.

Issue	Impact	Recommendation
No authentication	Any reachable user can upload, delete, chat, and read files.	Add auth before network deployment.
Permissive CORS	Browser-origin risk if exposed.	Restrict origins to configured frontend URLs.
No upload size limit	Denial-of-service risk through large uploads.	Enforce maximum size and stream file writes.
Unsanitized HTML rendering	XSS risk from user input, model output, filenames, or PDF content.	Sanitize Markdown output and avoid raw <code>innerHTML</code> for untrusted content.
Debug endpoint exposed	Expensive operations and tracebacks are available without auth.	Remove <code>/test-ingest</code> or guard with debug-only configuration.

14.2 Code Quality Recommendations

- Add `__init__.py` files under `app/`, `app/core/`, `app/llm/`, and `app/rag/` for clearer package semantics.
- Move duplicated `vector_to_sql()` into a shared utility.
- Replace ad hoc `test_db*.py` scripts with pytest tests.
- Add structured logging around ingestion start, chunk counts, provider selection, and retrieval results.
- Normalize uploaded filenames and persist both display title and storage filename.
- Replace character-based chunk sizes with tokenizer-aware chunking.
- Add status tracking for background ingestion so the frontend can distinguish uploaded, processing, failed, and ready papers.
- Add migrations through a migration tool such as Alembic rather than embedding growing SQL strings in startup code.

14.3 Architecture Recommendations

- Make AI mode a first-class request/session/paper property rather than a global mutable process setting.
- Introduce a service layer between routers and RAG/database modules to keep route handlers thin.
- Store embedding provider and model metadata with each chunk to prevent incompatible retrieval after mode or model changes.
- Add a task queue for ingestion if uploads become large or multi-user.
- Add RAG evaluation fixtures with known papers, expected citations, and answer-quality checks.

Chapter 15

Conclusion

MEZZINE_AI Research Assistant is a strong local MVP for AI-assisted research-paper analysis. It already implements the core RAG loop, multimodal PDF ingestion, chat streaming, persisted sessions, provider abstraction, and a polished static UI. The codebase is organized into sensible modules and is easier to maintain than a monolithic prototype.

The main next step is not adding more features, but making the existing architecture reliable and safe. Fixing dependency reproducibility, mode consistency, file/citation contracts, upload limits, CORS, debug exposure, and test coverage would materially improve the project. After those changes, the system would be well positioned for a more formal evaluation layer, user authentication, and production-ready deployment.

References

Bibliography

- [1] Local project file: `README.md`.
- [2] Local project file: `docs/architecture.md`.
- [3] Local project file: `backend/requirements.txt`.
- [4] Local project file: `backend/init.sql`.
- [5] Local source tree: `backend/app/` and `frontend/`.
- [6] Local design note: `privacy_mode_local_llm_feature.md`.
- [7] Local review note: `PROJECT_CRITIC_REVIEW.md`.